How Much Entity Integrity Matters [Experiments & Analysis]

A Methodology to Quantify the Impact of Entity Integrity Faults

Author names suppressed due to double-blind reviewing

ABSTRACT

Entity integrity is the principle that every entity of an application domain ought to be represented uniquely within a database. This is operationalized by using candidate keys, including the primary key. In practice, candidate keys are turned off to ease data acquisition, or the same entity is assigned multiple identifiers. Each of these malpractices can result in entity integrity faults. Despite its fundamental importance and despite the broad agreement that high quality analytics requires high quality data, we are unaware of studies that quantify the impact of entity integrity faults on analytical tasks. For this reason, we propose a methodology that enterprises can use to systematically quantify the impact of entity integrity faults on database queries. The methodology explores different dimensions of data quality, such as uniqueness and completeness, but also metrics that quantify the impact of violations on the precision, recall and performance of database queries. As a proof of concept, we apply the framework to the TPC-H benchmark. This illustrates how much entity integrity matters.

CCS CONCEPTS

• **Information systems → Relational database model**; **Integrity checking**; **Inconsistent data**; **Incomplete data**; **Structured Query Language**; **Database performance evaluation**.

KEYWORDS

Entity Integrity, Error Propagation, Experiments, Key, Query Accuracy, TPC-H

ACM Reference Format:

Author names suppressed due to double-blind reviewing. 2026. How Much Entity Integrity Matters [Experiments & Analysis]: A Methodology to Quantify the Impact of Entity Integrity Faults. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26), May 31 - June 6, 2026, Bengaluru, India.* ACM, New York, NY, USA, 13 pages.

1 INTRODUCTION

Data is the new oil[1]. This quote is one example how much importance organizations attribute to data in the 21st century, in an attempt to implement data-driven decision making, extract actionable insight, and gain competitive advantage. However, the extent of these benefits is limited by the quality of the data [35]. One of the most important principles, necessary to guarantee high quality data, is data integrity. In relational databases, entity integrity is operationalized by the use of primary keys. Here, each table has a single, non-null primary key to uniquely identify every record. This was already outlined by Codd when he proposed the relational model of data [19]. More generally, entities of an application domain are captured by records in a table, and candidate keys constitute minimal sets of table columns such that no entity has any non-null value on any column of the key, and no two different entities have values matching on every column of the key. The primary key of a table is a distinguished candidate key. Not using any candidate key means that tables may contain duplicate records, or miss information that makes it impossible to identify entities uniquely. This will result in database instances that misrepresent the current state of an application domain, which will produce faults in downstream tasks such as analytics. In terms of database queries and any other data-driven analytical methods such as AI, machine learning and statistics, results may be inaccurate, incomplete, or inconsistent, causing poor decision making, operational inefficiency, loss of trust and revenue, compliance risks, and waste of resources.

Despite its importance, entity integrity is not always guaranteed in practice. This may be due to the nature of the underlying data, or even be on purpose to ease with the acquisition of data or improve operational efficiency. Even when primary keys are being used, several issues can occur. Scenarios, such as a customer with multiple customer ids, are far from uncommon. Reliance on surrogate keys (artificial identifiers) to serve as the primary key without the use of any natural candidate key (also called business keys) can lead to recording the same real-world entity multiple times, each time with a different artificial identifier. The principle of entity integrity can even be abused, and situations where the same person has several social security numbers to claim benefits multiple times are possible. Furthermore, the sole focus on primary keys for entity integrity management poses challenges when integrating data from various sources across the same domain. For example, the artificial identifiers of products might not align across different datasets. Lastly, for big data applications that require horizontal scaling, facilitated through distributed systems, synchronizing the parallel generation of surrogate key values can become problematic.

Entity integrity can be initiated through robust schema design, and its efficient maintenance be implemented by the effective generation of any artificial identifiers and safe updates of values on every business key. This requires continuous monitoring in a dynamic, ever evolving environment that will challenge database practitioners. Due to their role in implementing entity integrity, keys are essential for all data models. They are fundamental in many classical and modern areas of data management. Knowledge about keys enables us to (i) uniquely reference entities across data repositories, (ii) minimize data redundancy at schema design time to process updates efficiently at run time, (iii) provide better selectivity estimates in cost-based query optimization, (iv) provide a query optimizer with new access paths that can lead to substantial speed-ups in

[1] https://sheffield.ac.uk/cs/people/academic-visitors/clive-humby

SIGMOD '26, May 31 - June 6, 2026, Bengaluru, India

Table 1: Tables with Entity Integrity Problems

(a) Table City

zip	city
1001	Shelbyville
1001	Shelbyville
NULL	Springfield

(b) Table Customer

cust_id	name	address	zip
1497	Homer	742 Green Rd	555
1824	Homer	742 Green Rd	555
1962	Montgomery B	1000 Ma Lane	1001

query processing, (v) allow the database administrator (DBA) to improve the efficiency of data access via physical design techniques such as data partitioning or the creation of indexes and materialized views, and (vi) provide new insights into application data. Modern applications raise the importance of keys further.

In spite of all of this, to our surprise, there is no methodology that quantifies the effects of entity integrity violations. In particular, to the best of our knowledge, no prior work has focused on how the accuracy of anticipated query workloads is impacted by entity integrity faults. This is particularly surprising as thought leaders in data quality research have called for work that quantifies the impact of data quality problems [37].

In view of the fundamental importance of entity integrity and very common malpractices, we establish a methodology that will enable enterprises to quantify the impact of entity integrity faults on database workloads. A proof of concept will be provided by applying the framework to the well-known relational TPC-H benchmark.

We investigate to which degree query accuracy of the 22 benchmark queries reduces when introducing either of the following entity integrity violations. First, we will investigate the effect that the duplication of records has. An example of such a violation is provided in Table 1a with primary key *zip* not being enforced: the city record with *zip* 1001 is repeated. Asking for all records within a given *zip* range may return duplicate records, and using `distinct` is slow due to a `unique` index not being available as the primary key is not enforced. Secondly, we analyse the impact of missing values on primary key attributes. An example can also be seen in Table 1a where the city record relating to 'Springfield' carries no value on *zip*. Asking for the name of cities within a given *zip* range that features the actual *zip* value of 'Springfield' will not return 'Springfield'. Finally, we analyse how queries are affected when primary keys are enforced, yet entity integrity is violated. We refer to such scenarios as deep entity integrity violations and an example can be seen in Table 1b. Here, the customer 'Homer' refers to the same entity but has been recorded as different customers since their primary key values on *cust_id* are different. Here, the composite business key {*name*,*address*} is not enforced, making it possible to assign different customer ids to the same customer. Attempting to retrieve all orders from the customer with *cust_id* '1497' will only give partial results in case orders from 'Homer' at '742 Green Rd' are meant.

Contributions: As these examples represent common integrity pitfalls, it is our aim to quantify how they impact the accuracy of queries. The contributions of our research can be summarized as follows.

- We establish a methodology for quantifying the impact of entity integrity faults on database workloads.
- The extent of faults will be measured in terms of the uniqueness and completeness requirements for primary keys, but also by the violation of alternate business keys.
- The impact on database workloads is measured by precision, recall, and efficiency of operations.
- It is shown how the size of data affects the impact.
- The effectiveness of the methodology is analysed experimentally on the 22 queries of TPC-H benchmark.

Our results emphasize the importance of using database keys for maintaining entity integrity, in particular the use of primary and alternate business keys. Enterprises are encouraged to apply our methodology to their databases, which will quantify the impact any entity integrity faults have on any of their database operations. The ability to quantify the impact can determine the level of investment in data governance to maximise the likelihood that expectations in high quality analytical outcomes can be met.

Organization: We discuss related work in Section 2. Afterwards, we present our methodology in Section 3. In Section 4, as a proof of concept, a comprehensive experimental analysis showcases how our methodology is applied, and what insights it can reveal. Finally, we conclude our work in Section 5 and discuss future research.

2 RELATED WORK

We position our contribution as a motivator for more research and development into data governance tools that facilitate high quality analytical outcomes and data-driven decision making. Likewise, organizations can apply our methodology to monitor how the entity integrity of their data affects the quality of their operations, and address concerns accordingly. To understand how our work is positioned within the rich literature, we will now review related work on data integrity in particular, data quality in general, as well as database repairs and consistent query answering.

2.1 Data Integrity

Since the very beginning of Codd's relational model of data [19], data integrity has always been a cornerstone for enabling fundamental data management tasks. While research has surfaced more than 100 classes of data integrity constraints [42], there are only three classes that enjoy native support within database systems. These comprise domain constraints, key constraints, and foreign key constraints, addressing Codd's three fundamental data integrity rules: domain integrity, entity integrity, and referential integrity [20]. Entity integrity is based on the principle that every entity of the application domain is represented uniquely within the database system. Traditionally, the principle is enforced by the concept of a database key, which is a set of attributes such that no two different records must exist in a database instance that have values matching on all the attributes of the key [20, 21]. Primary keys are distinguished database keys that provide us with a `unique` index, ensuring that records can be accessed quickly using the values of the record on the primary key. No column of a primary key must feature any null marker occurrence, which ensures that every entity can be accessed efficiently [20]. Due to their role in implementing entity integrity, keys are essential for all data models, including relational [31], conceptual [43], object-relational [25], XML [16], RDF [29], temporal [45], spatial [23], probabilistic [15],

and graph [3] models. Application of database keys encompass traditional data management areas such as data modelling, database design, indexing, and query optimization, but extend to modern tasks such as data integration [17], duplicate detection [34] and entity resolution [18, 41], database repairing and consistent query answering [4, 9], data cleaning [14, 22], data profiling [2], and the reliable use of large language models and AI agents for data processing [30]. Most existing work focuses on methods to enforce data integrity or detect them.

There is a need to quantify, through experiments, how different types of entity integrity violations affect the results of various types of queries [8, 35]. To conduct such comprehensive studies we first need to outline a methodology that guides this process. Applying this methodology can then be used to reveal which violations are most inhibiting to specific application workloads.

2.2 Data Quality

In our work, we investigate different types of entity integrity faults, comprising the duplication of records, missing values on primary key columns, and violations of business keys. Interestingly, these types of violations form the very essence of important data quality dimensions, namely uniqueness, completeness, and consistency, respectively [36, 40, 44]. In fact, quantifying the impact of entity integrity faults on database workloads on a regular basis will also motivate support towards upholding the important currency and timeliness dimensions of data as well. Frameworks for assessing the quality of data evolve regularly [6, 33, 38], demonstrating how important and challenging advances in improving data quality are.

The very reason data quality research has introduced different dimensions and various measures for quantifying the quality of data with respect to these dimensions is to help organizations answer the question whether their data is fit for purpose [7]. The methodology we propose aims at quantifying how fit the data is for processing data workloads in terms of their level of entity integrity.

Any new analytical movement, such as data warehousing [5], big data [39], data science [26, 47] or data-centric AI [1, 13, 49] comes back to the garbage in, garbage out principle: any insight brought forward by analytics can only ever be as good as the quality of the data the analytics is applied to. Our methodology will make it possible to quantify how the quality in terms of entity integrity affects the quality of database queries.

2.3 Repairs and Consistent Query Answering

The area of consistent query answering has originated from the very observation that data integrity faults cause incorrect answers in queries [4, 9]. The basic idea is to return only those answers that are certain to occur in every repair of the integrity faults. A repair of inconsistent data emerges from minimal changes to the data that restore consistency. The definition depends on what types of operations are permitted to change the data, such as insertions, deletions or modifications of data values or records [11]. The approach makes it possible to execute database workloads with confidence despite inconsistent data [10], which becomes necessary when it is unclear which repair is the right one. In this sense, consistent answers miss all of those query answers that would be returned on the right repair but not on other repairs. From this point of view, our methodology provides an analysis of what query answers could be missing when only certain ones are available. Note that the area of database repairs and consistent query answering has seen a prolific trajectory in terms of theory [12, 46], in particular recent work on primary and foreign keys [24, 27, 28], but tools and practical systems have not been brought forward to our knowledge.

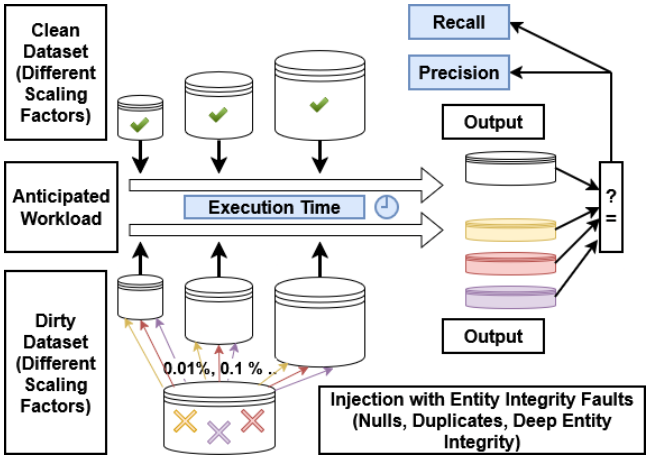

Figure 1: Illustration of Methodology

2.4 Summary

A methodology for quantifying the impact of entity integrity faults on database operations not only addresses the core for a rich research landscape on data integrity, data quality, data repairs and consistent query answering, but provides motivation for investment in further research and development to avoid or repair such faults, compute consistent answers, and build systems that facilitate the ultimate goals of high quality analytical insight and data-driven decision making.

3 METHODOLOGY FOR IMPACT ASSESSMENT

The main focus of our work is to provide a methodology that quantifies the impact of entity integrity faults on query accuracy. Figure 1 illustrates our proposal which we will describe now in detail.

Our methodology is based on several inputs. Firstly, we choose the dataset that will be investigated. In our case we have chosen the TPC-H benchmark for analysis. Secondly, we specify a workload that will be representative for what can be anticipated at the time. Results from analysing query logs could be harnessed to identify such a workload, for example. In our work we use the 22 TPC-H benchmark queries. Thirdly, we need to decide which dimensions of entity integrity faults we want to evaluate. These are determined by the different causes of such faults. Here, we focus on the following three causes. Firstly, we investigate how *duplicate records* affect query accuracy. Secondly, we analyse the effect of *missing values* on primary key attributes. These two cases cover traditional problems related to entity integrity that primary key enforcement aims to alleviate. The third cause acknowledges the fact that surrogate keys alone cannot guarantee entity integrity. For that reason, we also measure the impact of what we call *deep entity integrity violations*

Table 2: Answers on Original and Dirty Datasets

(a) Original Answers

orderpriority	order_count
1-URGENT	10781
2-HIGH	10484
3-MEDIUM	10366
4-NOT SPECIFIED	10587
5-LOW	10781

(b) New Answers

orderpriority	order_count
1-URGENT	11236
2-HIGH	10484
3-MEDIUM	10366
4-NOT SPECIFIED	11451

on query accuracy. Here, real-world entities occur multiple times in the database because different artificial identifiers are used to represent them. All three causes are common cases of malpractice [32], and represent inhibitors to the popular data quality dimensions of uniqueness, completeness, and inconsistency, respectively. For each of the three causes, we choose different degrees of severity based on a relative amount of violations compared to the database size. Moreover, we highlight in which tables of the database such violations occur. Finally, we also analyse how much the underlying database size matters for our analysis. In our case, this is naturally covered by the different scaling factors that apply to the TPC-H benchmark.

We have discussed the fundamental setting of our methodology. Now, we introduce the metrics to measure query accuracy. We run the dedicated workload on the original dataset and the dirty dataset that exhibits entity integrity violations. For each query we capture the output in both cases and compute recall and precision of the query. *Recall* measures how many of the clean query answers have been preserved, more precisely, recall takes the number of records that are part of both query answers on the clean and dirty dataset and divides it by the number of query answers on the clean dataset. *Precision* measures how many of the new query answers are actually clean, more precisely, precision takes the number of records part of both query answers on the clean and dirty dataset and divides it by the number of query answer on the dirty dataset.

An example is given by Table 2 where the records in yellow are both query answers on the original and dirty datasets. As there are 5 original answers, recall is 0.4 or 2/5; and since the new query has 4 answers, precision is 0.5 or 2/4.

Another important measure for database operations is performance. Hence, we also measure the *execution time* by which queries are evaluated. Since candidate keys, in particular the primary key, come with `unique` indices, query answering on the original dataset is supported by these structures.

4 EXPERIMENTS

Our experiments provide a proof of concept for our methodology. This is accomplished by quantifying the effects of entity integrity faults on the TPC-H workload. More precisely, we analyse how much violations of a primary key, such as duplicate records or occurrences of null, influence the accuracy of queries. For these scenarios we also measure potential performance gains that can be made through index structures that result from the presence of primary keys. Subsequently, we analyse deep entity integrity

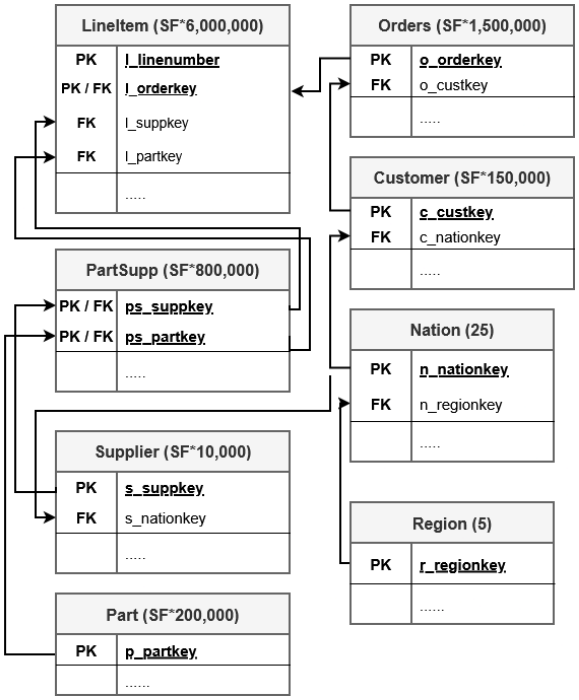

Figure 2: Database Schema of the TPC-H Benchmark

violations where entities are duplicated by assigning different primary key values. That is, primary keys are valid while some other business keys are violated.

4.1 Dataset, Set Up and Measures

We use the popular TPC-H benchmark[2] for RDBMs. Its database schema is shown in Figure 2, reduced to primary key (PK) and foreign key (FK) columns. For each table we see how the scaling factor (SF) determines how many records are generated. We used instances for three different scaling factors: small (0.01), medium (0.1) and large (1).

For each scenario in our experiments, every query has been run 100 times on each of the small and medium instance, and 50 times on the large instance. In each run, the time to execute the query has been recorded as well as its recall and precision. We refer the reader to the Github repository under https://anonymous.4open.science/r/entity_integrity_faults_impact-E26E for further details on the experiments and artifacts. We used MySQL 8.0.40 along with Python 3.12.2. For the hardware specifications we refer to Table 3.

4.2 Performance

We measure the performance gains achieved by `unique` indices associated with the primary key on each table. Such an index will automatically be created for every primary key and `unique` constraint. We measured the average execution time of each query run on the dataset, with and without index. From these results we recorded the improvement in execution time. Figure 3 shows the performance gains (in percent) for each query, and for each

[2]https://www.tpc.org/tpch/

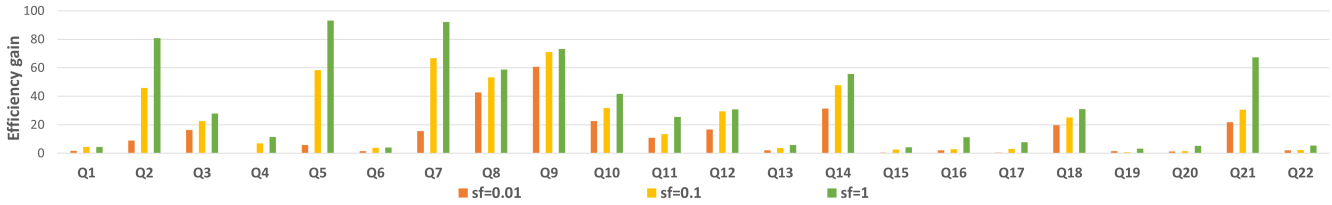

Figure 3: Efficiency Gains for Queries through Primary Key and Associated Index Structure

Table 3: Hardware Specifications for Experiments

Component	Details
CPU	Intel(R) Core(TM) i7-10610U CPU 2.30 GHz
Cores	4 (8 threads)
Memory	64 GB (DD4 - 3200 SODIMM)
Disk	2 TB (PCIe(R) Gen4 NVMe(TM) 2280 M.2 SSD)
System type	64-bit

of the three scaling factors. These results show the improvements achieved on average. More precisely, for each table of TPC-H we run queries each time with and without the primary key index. The data points in Figure 3 show the average across all of these runs.

It is evident that the gains in performance grow with the size of the instance. Indeed, queries take uniformly more processing time without the primary key index. Unsurprisingly, how much performance is gained by the index depends on the query. Q1, for example, has barely any gain. For Q2, however, performance gains range from noticeable gains on the small instance to over 40% gains on the medium instance and around 80% on the large instance. This means that, on average, the presence of a single index reduces execution time to one fifth of what is required without it.

Execution times fluctuate between different runs. For this reason we focus on the large instance since fluctuations have only marginal impact on the overall execution time. Large performance gains can be observed for queries Q2, Q5, Q7, Q9 and Q21 while moderate gains can be noted for Q3, Q8, Q10, Q12, Q14 and Q18.

To get a better idea on which table an index is particularly relevant for a query we have detailed results in Table 4. Here, we have listed the performance gains from a particular index (Table) for each query for all scaling factors (SF). To only list noticeable performance gains we limited results to a change in query execution time of at least 5%. Across all scenarios we notice how benefits increase with the size of the instance. Performance gains for queries Q2, Q5, Q7 and Q21 are driven by an index on the *supplier* table. For Q2 we notice that heavy computational effort is required for joins of the *partsupp* table with *part* and *supplier* to determine the cheapest supplier within a region for certain parts. Here, the index on *partkey* in *part* and the one on *suppkey* in *supplier* are prime candidates to enhance query execution. The *supplier* table is larger than the *part* table and the effect becomes more pronounced there. In contrast, there are no significant speed-ups from the index on the *part* table. Unlike Q2, the queries Q5, Q7 and Q21 do not depend on the *part* table at all. Further investigation of Q9, where for each order year all qualifying records in *orders* must be joined with *lineitem*, shows great performance gains resulting from an index on *orders*.

For queries with moderate gains we see that Q3 benefits from an index on *lineitem*. For Q8 this is the case for *nationkey* in *nation*. Equally, this holds for Q10 and in addition index support on *customer* results in noticeable improvements there. For Q12 we notice that *orders* plays a crucial role in this regard and in the case of Q14 it is the *part* table that can provide moderate speed-up. For Q18 this seems more balanced and the benefits come from the index structure on *customer* and *lineitem*.

The remaining queries show only little improvement in execution time when introducing indices on primary keys. In particular, for Q1 and Q6 the introduction of an index on the primary key of *lineitem* does not have an effect at all. Both times the query only makes use of this table and only non-key attributes are filtered and used for aggregation. The other queries showing little performance gains are less join-heavy and look-ups on the primary keys do not occur or are limited. For details on the experimental results we refer the reader to our Github repository.

In summary, the introduction of primary keys and their index strongly improves query execution time. As our experiments have shown, not all queries benefit equally from these indices. Moreover, we have seen that for those queries that do show performance gains, these are often attributed to certain tables. Surprisingly, an index on the primary key of the largest table *lineitem* only results in small performance gains. However, a similar index on the smaller *supplier* table proves to be much more valuable in terms of performance. An index on the *region* table does not lead to any noticeable performance gains, which is not surprising as the table contains consistently five records. However, for the slightly larger table *nation*, which has always 25 tuples, an index on its primary key results in moderate query speed-up. This is likely due to the nature of the queries in the TPC-H workload: Some queries only search the *region* table once, while the *nation* table joins multiple times with other tables. Better performance is a welcome side effect when enforcing entity integrity and, in what follows, we will showcase the impact on query accuracy when entity integrity faults occur.

4.3 Accuracy

Our experiments aim at quantifying the loss in accuracy for analytical queries when different degrees of entity integrity violations occur. For this purpose, we investigate three kinds of violations.

4.3.1 Entity Duplication. First, we investigate the impact of duplicates on query accuracy. This means we duplicate records in a table, implying that the primary key for this table is turned off. For

Table 4: Effect of Index on Query Execution Speed of TPC-H Workload

Table	SF	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
region	0.01																						
	0.1																						
	1.0																						
nation	0.01		36.2%						86.9%	40.1%	33.1%												
	0.1		66.1%						90.3%	52.8%	47.7%												
	1.0		72.5%						92.1%	55.1%	58.3%												
customer	0.01										36.2%								18.9%				
	0.1										45.1%			6.4%					28.4%				
	1.0										57.7%			8.9%					35.0%				
orders	0.01			7.2%						88.7%			29.6%										
	0.1			13.3%				9.0%		92.5%			45.1%										
	1.0			18.5%				15.2%		93.2%			45.6%										6.7%
supplier	0.01					26.6%		37.9%		39.8%		10.1%										49.7%	
	0.1		69.6%			89.3%		90.4%		53.6%		12.1%										59.9%	
	1.0		94.7%			98.7%		98.2%		54.1%		12.6%				6.5%					6.2%	88.7%	
part	0.01														45.5%								
	0.1														64.9%								
	1.0														71.6%								
partsupp	0.01											17.6%											
	0.1											25.3%					7.0%						
	1.0		9.0%							6.2%		46.0%					19.9%						
lineitem	0.01			35.6%		11.6%		25.0%	7.8%		9.9%								33.9%			15.7%	
	0.1			43.7%	11.6%	17.4%		35.3%	11.2%		9.9%							8.7%	36.7%		8.4%	16.7%	
	1.0			48.2%	18.1%	23.5%		42.7%	12.6%		10.2%							16.0%	43.4%		11.7%	19.5%	

an example, we refer again to Table 1 where two city records are identical. In our experimental setup we have only introduced duplicates for one table at a time. This means we can easily locate from which source any impact of such violations on queries originates.

The impact of duplicates on the TCH-H workload is summarized in Table 5. For each table we have noted the percentage (0.01%, 0.1%, 1% and 10%) of records that we randomly duplicated. In case of the *region* and *nation* table we have introduced exactly one duplicate each time. Unlike the other tables, these two do not scale with the size of instances and always carry 5 and 25 records, respectively. It is worth noting that for the small instance and 0.01% duplication, the tables *customer*, *supplier*, *part* and *partsupp* are not affected as the percentage represents less than one record. The same applies to *supplier* for the small instance and 0.1% duplication, as well as for the medium dataset and 0.01% of duplicated entities. For each scaling factor (SF), Table 5 shows the mean and median value for recall (R) and precision (P) across all benchmark queries for different duplication factors.

We observe that a growing percentage of duplicates causes a drastic decline of query accuracy. What seems interesting is that for few entity integrity violations the median sits above the mean, which indicates that most queries provide the correct result and some outliers related to incorrect outputs bring down the average. However, for larger instances this trend seems to reverse with a higher number of duplicates. This means that highly accurate outputs become the exception. Furthermore, for the same number of violations, accuracy declines with growing scaling factor. This effect may be due to the fact that some parameters, such as the date range of order placements, remain the same while the instance becomes larger. So, with growing size of the instance and fixed duplication factor there is a higher likelihood to have a duplicated order within a certain date range, which might distort the query output. Similar effects may be observed with regards to violations in the dataset elsewhere.

Table 5: Impact of Duplicates on TPC-H Workload

		Percentage of Violations							
SF	mean/ median	0.01%		0.1%		1%		10%	
		R	P	R	P	R	P	R	P
0.01	mean	0.962	0.936	0.927	0.903	0.841	0.804	0.654	0.621
	median	1	1	1	1	1	1	0.883	0.8
0.1	mean	0.917	0.903	0.84	0.825	0.657	0.64	0.472	0.449
	median	1	1	1	1	0.963	0.96	0.404	0.277
1	mean	0.847	0.834	0.67	0.658	0.521	0.503	0.455	0.434
	median	1	1	0.996	0.992	0.908	0.8	0.358	0.05

Having provided a high-level overview, we now aim for a more detailed analysis. Going forward, we will fix the scaling factor of the dataset (sf = 1) and we keep the number of duplicates in corresponding tables at 0.1 percent. Now, we look at the effects of these integrity faults on recall and precision for each query. Figure 4 summarizes this scenario. Here, for each query, different sets of runs have been performed where, each time, one of the tables involved in the query contains duplicates and the others do not.

First, we notice that recall and precision are often very similar and, in fact, coincide for several queries. Whenever they coincide the queries return the same number of answers on the original and dirty dataset. Whenever they differ the duplicates do not only cause a change in some values of the query answers, but leads to whole new answers or missing out on some previous answers. Noticeable deviation in recall and precision can be observed for Q13 and Q18, while minimal differences are present in the case of Q2 and Q11. For other queries such differences are marginal. In Table 6 we summarized the average number of query answers on the original and dirty datasets. These numbers indicate that recall and precision do also not coincide for Q20. Again, these differences are marginal and seem to result from particular runs.

The TPC-H workload can be grouped into three general categories regarding the impact of duplicates on accuracy. Queries Q1, Q6, Q12 and Q14 are heavily affected, while Q2, Q3, Q10, Q11,

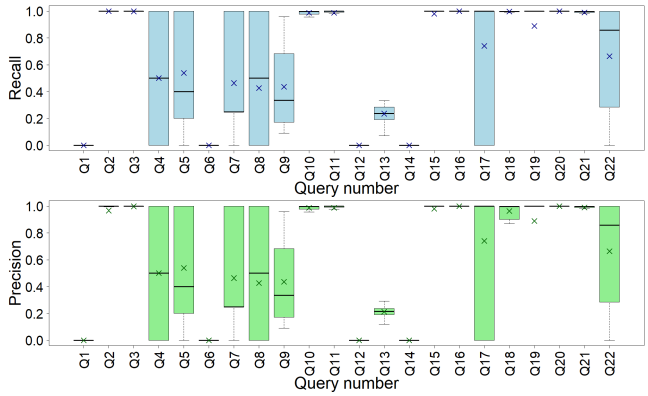

Figure 4: Recall and Precision of Query Execution Grouped by Query Number with respect to Duplicates

Q15, Q16 and Q18 - Q21 still provide highly correct outputs with the exception of Q19 which suffers slightly more than others. The remaining queries lie somewhere in between, exhibiting varying degrees of accuracy loss.

Now that we have discussed the effect of duplicates on different queries we take a look at how duplicate records affect the TPC-H workload with respect to where in the database they occur. This breakdown is illustrated in Figure 5. The different shading of the plots for the *region* and *nation* tables indicates that here, consistently one tuple has been duplicated, unlike 0.1 percent of tuples in the other tables.

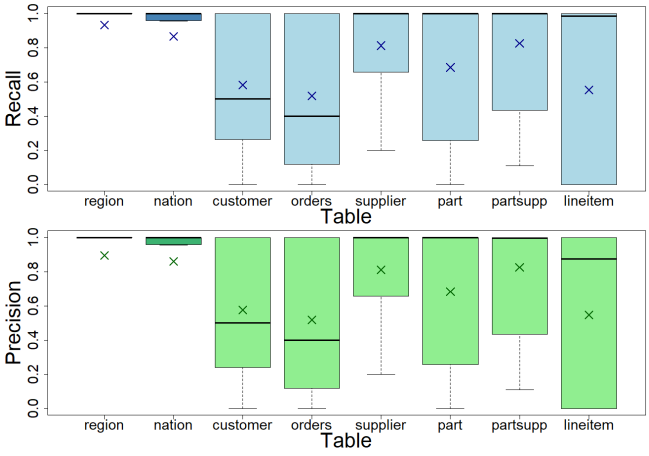

Figure 5: Recall and Precision of Query Execution Grouped by Tables with respect to Duplicates

First, we observe that there seems to be no fundamental deviation between recall and precision in general. Most notably, when introducing duplicates in the *lineitem* table, the median precision of queries is lower than the median recall, while the mean for both measures aligns. Duplicates in *region* and *nation* do not seem to have major effects. Although here, the mean below the box plot might indicate that some outliers drag the mean down. That is, if a query becomes affected, then heavily so. Duplicates in the other tables influence the respective queries of the TPC-H workload to varying degrees. In particular, the *customer* and *orders* table show low mean and median recall and precision. This means in those two tables it is particularly crucial to guarantee entity integrity, especially when it comes to duplicated records. However, the results for all tables, apart from *region* and *nation*, show quite a range in query recall and precision. Hence, without entity integrity, query outputs can become highly inaccurate. A possible explanation for these effects is the following. Many of the TPC-H queries aggregate information over some specific nations and regions. These queries still produce the correct output, even when duplicates exist in these tables. However, when counting the number of customers or the orders per customer with certain conditions, then duplicates in *customer* or *orders* can skew query answers considerably.

Overall, duplicates of a table affect all queries that use the table. Tab. 7 quantifies how much different percentages of duplication on some specific table at a time affect the recall of every query on average. As mentioned before, the number of duplicates for *region* and *nation* is consistently one. We have also seen that in most cases, precision and recall are only marginally different, if at all.

Tab. 7 shows that the recall of a query is affected quite dramatically by only modest percentages of duplication (%D), whenever duplication in a table has any impact on the query. In the following, we investigate this effect in a more nuanced manner. The majority of queries use some aggregation. Q1 and Q6 are affected significantly by duplication in *lineitem*, which is the only table involved in these queries. Q12 and Q14 exhibit similar behaviour across multiple tables that affect them. A slightly different picture emerges for Q4, Q8, Q15 and Q22. For Q8, duplicates in *region* have no effect at all. Indeed, the query filters by parts that satisfy some conditions and have been ordered by customers from some nation in some region. Hence, joining the *nation* and *region* table carrying some duplicated record will not yield a different result. This behaviour is mirrored by Q4 and Q22, yet there it is the *lineitem* and *orders* table respectively, that does not impact query accuracy. In both queries existence checks are conducted on the corresponding table, hence duplicates have no effect on the output. Q15 is impacted by duplicates in *lineitem*, but not by violations in *supplier*. The situation for Q17 and Q19 is comparable to Q12 and Q14, only that accuracy is not affected as heavily for the latter. Queries Q3, Q5, Q7, Q9, Q10, Q11, Q13, Q16, Q18 and Q21 show various degrees of decline in accuracy, depending on which table is affected by duplicates. Finally, duplicates seem to have no effect at all on Q16. Here, this is due to aggregating over distinct records.

For Q20 we notice that only duplicates in *lineitem* can impact this query, and even then a different output is highly unlikely. The query aggregates the quantity of *lineitem* records associated with particular parts and grouped by suppliers who supplied this part. Then the query checks if the supplier of these parts has enough capacity. As the difference between this capacity and the aggregated values from *lineitem* is quite substantial, most of the time even high levels of duplication in *lineitem* can only rarely distort results. On top of that, an additional filter that looks for suppliers in certain countries makes a change in output even less likely.

Finally, we analyse the only truly pointy query, Q2. Duplicates in any table might either not affect query Q2 at all or if they do, some suppliers (along with the corresponding nation name and part

Table 6: Average Size of Output from TPC-H Workload on Original and Dirty Dataset (with Duplicates)

Dataset	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
original	4	471.24	11428.89	5	5	1	3.84	2	175	37737.56	890.69	2	42	1	1	18266.69	1	803.23	1	173.65	375.38	7
dirty	4	497.61	11428.89	5	5	1	3.84	2	175	37737.56	891.03	2	46.16	1	1	18266.69	1	833.07	1	173.69	375.38	7

Table 7: Effect of Duplicates on Accuracy of TPC-H Workload with sf=1

Table	%D	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
region			1.0			0.8			1.0														
nation			1.0			0.948		1.0	0.04	0.96	0.96	0.952									1.0	0.978	
customer	0.01			0.999		0.912		0.866	0.91		0.999			0.702					0.999				0.877
	0.1			0.999		0.3		0.308	0.43		0.999			0.281					0.999				0.328
	1.0			0.989		0.0		0.0	0.02		0.99			0.147					0.99				0.0
	10.0			0.9		0.0		0.0	0.0		0.899			0.072					0.9				0.0
orders	0.01			0.999	0.356	0.912		0.883	0.84	0.827	0.999		0.21	0.393					0.999			0.999	1.0
	0.1			0.999	0.0	0.264		0.25	0.37	0.159	0.998		0.0	0.19					0.999			0.99	1.0
	1.0			0.99	0.0	0.0		0.0	0.0	0.0	0.987		0.0	0.07					0.989			0.906	1.0
	10.0			0.9	0.0	0.0		0.0	0.04	0.0	0.874		0.0	0.026					0.901			0.37	1.0
supplier	0.01		1.0			0.948		0.933	0.93	0.962		0.998				1.0					1.0	0.999	
	0.1		1.0			0.664		0.635	0.37	0.686		0.992				1.0					1.0	0.999	
	1.0		1.0			0.016		0.026	0.1	0.02		0.924				1.0					1.0	0.99	
	10.0		1.0			0.0		0.0	0.04	0.0		0.436				1.0					1.0	0.898	
part	0.01		1.0						0.92	0.894					0.0		1.0	1.0		1.0	1.0		
	0.1		1.0						0.41	0.329					0.0		1.0	0.86		0.9	1.0		
	1.0		1.0						0.0	0.0					0.0		1.0	0.18		0.34	1.0		
	10.0		1.0						0.04	0.0					0.0		1.0	0.1		0.0	1.0		
partsupp	0.01		1.0							0.888		0.999					1.0						
	0.1		1.0							0.326		0.992					1.0						
	1.0		1.0							0.0		0.921					1.0						
	10.0		1.0							0.0		0.419					1.0						
lineitem	0.01	0.015		0.999	1.0	0.848	0.0	0.893	0.94	0.824	0.999		0.19		0.0	1.0		0.94	0.999	0.98	1.0	0.999	
	0.1	0.0		0.997	1.0	0.236	0.0	0.245	0.34	0.161	0.997		0.0		0.0	0.92		0.62	0.993	0.88	1.0	0.989	
	1.0	0.0		0.973	1.0	0.0	0.0	0.0	0.0	0.0	0.97		0.0		0.0	0.34		0.12	0.933	0.18	1.0	0.9	
	10.0	0.0		0.767	1.0	0.0	0.0	0.0	0.04	0.0	0.742		0.0		0.0	0.0		0.22	0.479	0.0	0.999	0.369	

columns) will be recorded multiple times. However, the original output will always be retained by the new output. Hence, recall is never impacted. Tab. 6 provides further insight, and also explains the difference between recall and precision for Q2. For further details on the experiments we refer to our Github repository.

In summary, we have seen that duplicate records, as one type of entity integrity faults, can result in highly inaccurate queries. Our experiments highlighted which tables need to be kept duplicate-free the most in terms of retaining the accuracy of queries from the TPC-H framework. Duplication in *region* and *nation* have small impact, as expected. In fact, duplicate records in these tables are not propagated strongly along child tables. The remaining tables carry more weight in this regard. The effects of entity integrity violations are multi-faceted, and each query presents a distinct pattern of impact by duplication faults in the tables involved. In addition, the parameters passed to queries also inform to which degree query accuracy might be impaired. Furthermore, we outlined how even comparatively small levels of duplication can lead to severe query inaccuracy. On top of this, some queries not only produce outputs with incorrect values, but their outputs could even miss previous answers or introduce new ones. In addition, we have demonstrated that under a fixed percentage of duplication, the accuracy drops more when the size of the instance grows.

4.3.2 Null Violation. In this experiment, primary keys were turned off for those TPC-H tables where we introduced null markers in some columns of the key. Table 1 shows a city record in which the "zip" column is NULL.

Table 8: Impact of Null Violations on TPC-H Workload

		Percentage of Violations							
SF	mean/ median	0.01%		0.1%		1%		10%	
		R	P	R	P	R	P	R	P
0.01	mean	0.971	0.951	0.954	0.933	0.893	0.874	0.692	0.695
	median	1.000	1.000	1.000	1.000	1.000	1.000	0.889	1.000
0.1	mean	0.944	0.936	0.884	0.875	0.701	0.695	0.532	0.542
	median	1.000	1.000	1.000	1.000	0.978	0.998	0.826	0.842
1	mean	0.882	0.873	0.711	0.704	0.581	0.576	0.518	0.529
	median	1.000	1.000	0.997	0.999	0.961	0.982	0.836	0.834

First we show how the presence of null violations affects precision and recall of the queries in TPC-H. A summary is shown in Table 8. Again, we have performed experiments with different percentages of integrity faults in each table, and for *region* and *nation* we consistently introduced one record with null violations.

The results in Table 8 are similar to those in Table 5. Increasing null violations in the dataset leads to lower query accuracy. Similarly, larger instances result in larger query accuracy loss under a fixed percentage of null violations. Comparing the results in Table 8 and Table 5, we notice that duplicates have a larger impact than nulls. This is particularly true when more errors are introduced.

Just like before we fix the TPC-H scaling factor to 1, and the percentage of null violations in relevant tables to 0.1%. Fig. 6 shows

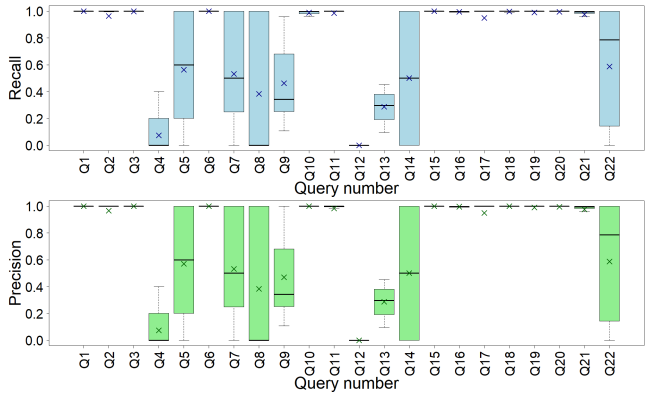

Figure 6: Recall and Precision of Query Execution Grouped by Query Number with respect to Null Violations

a breakdown of recall and precision for each query, with respect to null violations across different tables that are involved in the query. As before, we do not account for combinations of particular database scaling factors and particular percentages of violations in particular tables whenever these affect less than one record on average.

Fig. 6 surfaces only marginal differences between recall and precision for each query, with Q10 displaying the largest difference. Tab. 9 shows the precision and recall per table. This is mostly in line with the effects of duplicates on query accuracy. However, when comparing Tab. 9 and Tab. 6 we notice that query recall and precision differ more under null violations than they differ in the presence of duplicates. Most notably, Tab. 9 shows that null violations often lead to queries with fewer answers than they should. Hence, there is a decrease in recall. For most queries this will be due to joins not being performed because values of the key are missing. This represents a major difference from how duplicates affect queries.

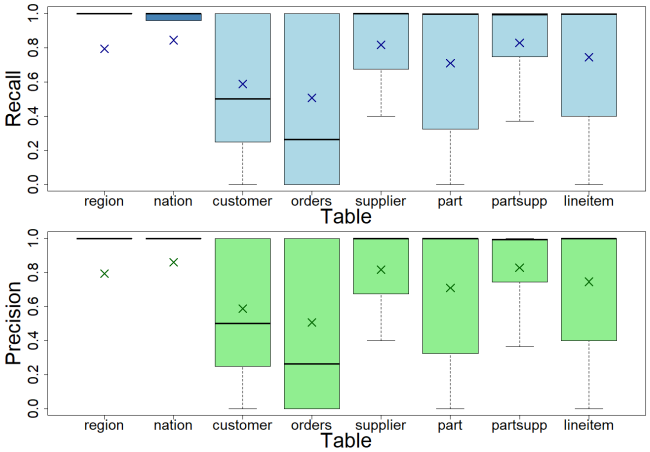

Figure 7: Recall and Precision of Query Execution Grouped by Tables with respect to Null Violations

We now proceed with a more detailed analysis, and investigate the effect of null violations with respect to the tables that exhibit these violations. Fig. 7 shows only marginal differences between precision and recall. Altogether, the breakdown into tables shows that results for violations by duplicates in Fig. 5 are similar to those by nulls.

The following provides a detailed breakdown of how each query is affected by the percentage of null violations (%N), and how these violations in a relevant table affect the recall of query answers. This has been done in a similar manner as before, with the results illustrated in Tab. 10. Comparing this table to Tab. 7, duplicates and null violations have similar effects on query accuracy, in terms of which tables cause the inaccuracies and how pronounced they are. One of the most noticeable differences is the impact of null violations in the *lineitem* table. Here, we see that Q1, Q6, Q14, Q17 and Q19 are not affected by null violations, while duplicates have an effect on these. This is likely due to the fact that none of these queries exhibit joins with the *orders* table, and do not make use of the primary key in the *lineitem* table in any other way. In the case of Q4 we observe the opposite behaviour. Here, null violations in *lineitem* have an effect as we join this table with *orders*. However, duplicate records in the *lineitem* table have no effect as the query investigates orders that have been placed, comprised of lineitems with certain conditions. Hence, a duplicated record in *lineitem* does not impact this filtering condition. Furthermore, null violations in any of the tables involved in Q2 lead to changes in recall, unlike the previous case of duplicate records. The reason for this is likely that, in the presence of null violations, some of the joins in Q2 cannot be executed correctly. Similarly, this seems to be the case for Q16. Query Q8 is also affected by violations in the *region* table this time. This query aggregates over *orders* associated with a certain *nation* parameter, and links this to other nations associated with the *region* of the given *nation*. Here, duplicates in the *region* table would have no effect, yet null violations would prevent establishing which *region* is associated with the provided *nation*. The behaviour of Q7 and Q20 paints a similar picture with respect to *nation* and *region*. Unlike duplicates in the *supplier* table, null violations affect the joining of the *lineitem* table with the *supplier* table in Q15. This is also the case for Q16 and Q20 and the corresponding tables there. Again, detailed information on our experiments and results can be found in our Github repository.

Overall, missing primary key values can severely impact query accuracy, often leading to missing or incorrect records, especially when compared to duplicates. Interestingly, the impact of null violations on query accuracy is not more severe than the impact of duplicates. In particular, when primary key values are missing, joins are affected in scenarios where duplicates have no impact. On the other hand, queries that aggregate over tables, but do not involve its primary key columns, are not affected by null violations, compared to duplication of records. We reiterate that enforcing primary key constraints ensures entity integrity and can improve query performance by benefiting from auto-indices.

4.3.3 Deep Entity Integrity Violation. In this part of our experimental study, we examine the impact of deep entity integrity violations, where the same entity is represented multiple times, each time with

Table 9: Average Size of Output from TPC-H Workload on Original and Dirty Dataset (with Null Violations)

Dataset	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
original	4	472.05	11451.65	5	5	1	3.66	2	175	37667.225	922.32	2	42	1	1	18273.25	1	793.48	1	170.77	377.99	7
dirty	4	455.18	11443.16	5	4.74	1	3.57	1.94	173.83	37271.14	912.23	2	42	1	1	18268.07	1	791.67	1	169.80	371.68	7

Table 10: Effect of Null Violations on Accuracy of TPC-H Workload with sf=1

Table	%N	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
region			0.738			0.76			0.74														
nation			0.917			0.94		0.976	0.02	0.96	0.959	0.968									0.941	0.958	
customer	0.01			0.999		0.9		0.907	0.92		0.999			0.791					0.999				0.845
	0.1			0.999		0.368		0.391	0.37		0.999			0.381					0.998				0.174
	1.0			0.99		0.0		0.0	0.06		0.99			0.196					0.99				0.0
	10.0			0.9		0.0		0.0	0.06		0.899			0.085					0.899				0.0
orders	0.01			0.999	0.32	0.844		0.843	0.95	0.829	0.999		0.25	0.392					0.999			0.998	1.0
	0.1			0.999	0.0	0.232		0.232	0.26	0.169	0.998		0.0	0.193					0.999			0.99	1.0
	1.0			0.99	0.0	0.0		0.0	0.04	0.0	0.987		0.0	0.064					0.989			0.908	1.0
	10.0			0.899	0.0	0.0		0.0	0.02	0.0	0.874		0.0	0.002					0.9			0.369	1.0
supplier	0.01		0.999			0.936		0.949	0.94	0.963		1.0				1.0					0.999	0.999	
	0.1		0.998			0.672		0.744	0.29	0.68		0.998				1.0					0.999	0.998	
	1.0		0.99			0.02		0.01	0.08	0.016		0.987				0.98					0.99	0.99	
	10.0		0.902			0.0		0.0	0.02	0.0		0.869				0.84					0.9	0.901	
part	0.01		1.0						0.86	0.887					0.02		0.999	1.0		0.98	0.999		
	0.1		0.999						0.48	0.347					0.0		0.998	0.9		0.98	0.999		
	1.0		0.99						0.04	0.0					0.0		0.983	0.48		0.22	0.992		
	10.0		0.902						0.0	0.0					0.0		0.845	0.2		0.0	0.922		
partsupp	0.01		0.999							0.889		1.0					0.999						
	0.1		0.998							0.326		0.998					0.993						
	1.0		0.99							0.0		0.985					0.937						
	10.0		0.904							0.0		0.867					0.531						
lineitem	0.01	1.0		0.999	0.844	0.904	1.0	0.875	0.91	0.88	0.999		0.34		1.0	1.0		1.0	0.999	1.0	1.0	0.997	
	0.1	1.0		0.998	0.148	0.416	1.0	0.393	0.46	0.289	0.998		0.0		1.0	1.0		1.0	0.995	1.0	1.0	0.976	
	1.0	1.0		0.982	0.0	0.0	1.0	0.0	0.08	0.0	0.98		0.0		1.0	1.0		1.0	0.957	1.0	1.0	0.791	
	10.0	1.0		0.84	0.0	0.0	1.0	0.0	0.04	0.0	0.822		0.0		1.0	1.0		1.0	0.621	1.0	1.0	0.255	

a different primary key value. In general, this is possible because some business key is violated.

Table 11: Impact of Deep Entity Integrity Violations on TPC-H Workload

		Percentage of Violations							
SF	mean/ median	0.01% R	0.01% P	0.1% R	0.1% P	1% R	1% P	10% R	10% P
0.01	mean	0.994	0.985	0.977	0.962	0.943	0.922	0.863	0.842
	median	1	1	1	1	1	1	1	1
0.1	mean	0.98	0.974	0.952	0.938	0.892	0.882	0.83	0.822
	median	1	1	1	1	1	1	1	1
1	mean	0.956	0.944	0.9	0.891	0.856	0.844	0.815	0.809
	median	1	1	1	1	1	1	1	1

Table 1 shows an example of multiple customer records with different identifiers. Despite satisfying artificial identifier constraints, other meaningful keys may be violated, highlighting the need for business keys based on natural attributes for entities. Our experiments involve the duplication of entities with different identifiers in one table, and random reassignment of references to either identifier from records in the child tables. We refer to such a scenario using child and parent table names connected through an arrow, for example *customer* ← *orders*. The sole use of surrogate keys has shortcomings as our experiments highlight.

We have conducted experiments for *orders* ← *lineitem*, *customer* ← *orders*, *part* ← *partsupp* ← *lineitem* and *supplier* ← *partsupp* ← *lineitem*. In the last two cases we have propagated changes of the primary key value up to the *lineitem* table. These four scenarios present realistic situations in which, for example, the same customer is stored with multiple customer ids, due to simply using surrogate identifiers. Naturally, it then happens that some orders are attributed to one id, and other orders attributed to another id.

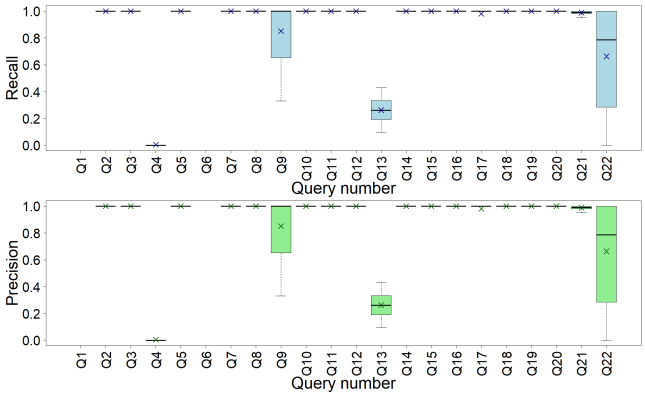

Figure 8: Recall and Precision of Query Execution Grouped by Query Number with respect to Deep Entity Integrity Violations

The effect of deep entity integrity violations on the TPC-H workload for different scaling factors is outlined in Table 11. While

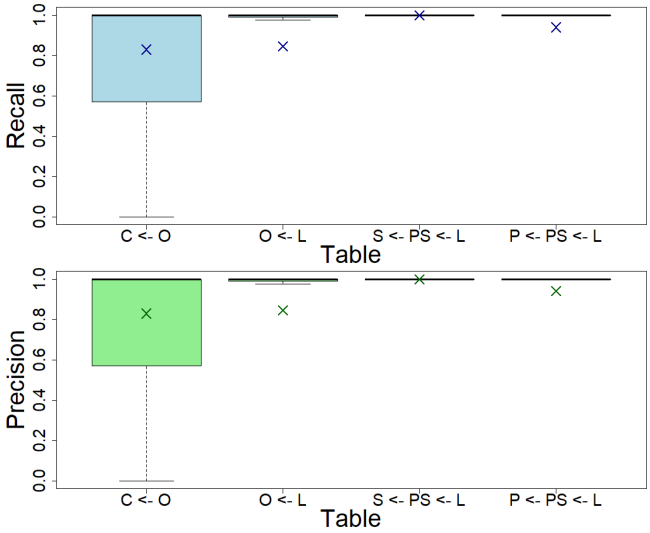

Figure 9: Recall and Precision of Query Execution Grouped by Tables with respect to Deep Integrity Violations

dataset size and violation frequency influence accuracy, deep entity integrity violations generally cause slower degradation compared to duplicates or null violations. For a more nuanced breakdown we have again fixed the scaling factor sf at 1 and considered deep entity integrity violations for 0.1%.

As before, we look at the effects of violations broken down for each query, which is shown in Figure 8. Results for Q1 and Q6 are excluded as none of the scenarios for which we conducted experiments affect these. Q4 is heavily affected, followed by Q13, Q22, Q9 and Q21. The dirty dataset does either not affect the remaining queries at all or only marginally. Under deep entity integrity violations, recall and precision seem to align more than in the previously discussed scenarios. This is supported by the results shown in Table 12. We note that the 'x' indicates that no outputs were recorded as the queries had not been affected.

We have further investigated the impact of violations based on the table they originate from (e.g., *customer* or *orders*). The results are shown in Figure 9. The most significant effects originate from *customer* and *orders* errors, with some outliers affecting query accuracy more in the latter. Violations in the *supplier* and *part* table, however, have less impact.

Table 13 provides a detailed breakdown of the impact of varying percentages of deep integrity violations (%V). The violation *supplier* ← *partsupp* has minimal effect on TPC-H accuracy, while other violations affect specific queries to varying degrees. Affected queries include Q3, Q4, Q9, Q10, Q13, Q17, Q18, and Q20-Q22.

The remaining queries fall into two categories. Some queries, like Q1, are unaffected as the relevant violations do not impact them. That is, *orders* ← *lineitem* does not impact foreign key columns in *lineitem*, the only table that Q1 uses. Others, like Q2, remain unchanged because the violations do not alter the results of the query. In this case, selection on tables and subsequent joins lead to the same answers with and without deep integrity faults.

However, Q13 experiences significant accuracy loss in both *orders* ← *lineitem* and *customer* ← *orders* scenarios, as incorrect aggregation leads to wrong outputs. This query looks for certain types of orders made by customers, and then returns the number of orders together with the number of customers that have placed this many orders. As either violation scenario leads quite likely to incorrect aggregation over the respective table, the output becomes incorrect. In contrast, Q10 shows minimal accuracy loss, with aggregation over *lineitem* still correct, though *customer* ← *orders* causes incorrect outputs sometimes. Here, aggregation over the *lineitem* table is still executed correctly in the scenario *orders* ← *lineitem*, as the query filters by certain *orders* and then sums up over all associated *lineitem* records. As the query returns the primary key value of customers along with other information, this results in incorrect outputs under *customer* ← *orders* violations. In this case, degradation of accuracy correlates with the percentage of violations. Results for Q3, Q4, Q9, Q17, Q18, Q20-Q22 follow similar patterns to the previously discussed scenarios. Details on all experimental results are available in the referenced GitHub repository.

We conclude that deep entity integrity violations can significantly reduce query accuracy, highlighting the importance of enforcing business keys alongside artificial identifiers. While their impact is generally less severe than duplicates or null violations, these errors may be more subtle and harder to detect and fix. The severity depends on the percentage of violations and database size, though the effect is more concentrated on specific queries, with recall and precision aligning better than in scenarios involving duplicates or missing primary keys. Despite these violations, primary keys are still enforced, and query performance benefits from existing index structures. Addressing these violations through business key constraints, supported by indexes on those attributes, can further improve query speed.

5 CONCLUSION AND FUTURE WORK

We have proposed a methodology for assessing the impact of entity integrity violations on workload accuracy across multiple data quality dimensions. It helps database practitioners understand by how much different types of entity integrity violations affect queries, including duplicate records, partial records and deep violations. It also identifies areas of the data repository that need attention to ensure query accuracy, and highlights where index structures can offer the greatest performance improvements. Using the TPC-H benchmark, we demonstrated that even small entity integrity violations can cause significant inaccuracy in analytical query answers. Our findings stress the importance of robust schema design with regard to entity integrity that incorporates business keys alongside artificial identifiers. Our framework aids decision-making in this design process.

The methodology provides a first step towards measuring the impact of data quality issues on data workloads, offering several avenues for future investigation. One potential direction is the application of our methodology to other relational benchmarks, which could provide further showcases into the impact of entity integrity violations. Additionally, an analysis of real-world datasets that already exhibit these violations could yield valuable findings. However, as noted by the authors in [37], using real-world datasets

Table 12: Average size of output from TPC-H workload on original and dirty dataset (with deep entity integrity violations)

Dataset	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
original	x	469.13	11453.26	5	5	x	3.71	2	175	37575.69	977.38	2	42.00	1	1	18270.68	1	806.35	1	168.67	391.5	7
dirty	x	469.13	11455.80	5	5	x	3.71	2	175	37578.29	977.38	2	42.08	1	1	18270.68	1	805.99	1	168.66	391.5	7

Table 13: Effect of deep entity integrity violations on accuracy of TPC-H workload with sf=1

Scenario	%V	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22
$c \leftarrow o$	0.01			1.0		1.0		1.0	1.0		0.999			0.62					0.999				0.914
	0.1			1.0		1.0		1.0	1.0		0.999			0.327					0.999				0.325
	1.0			1.0		1.0		1.0	1.0		0.994			0.179					0.995				0.0
	10.0			1.0		1.0		1.0	1.0		0.943			0.08					0.948				0.0
$o \leftarrow l$	0.01			0.999	0.556	1.0		1.0	1.0	1.0	1.0		1.0	0.384					0.999			0.998	1.0
	0.1			0.999	0.004	1.0		1.0	1.0	1.0	1.0		1.0	0.197					0.999			0.978	1.0
	1.0			0.992	0.0	1.0		1.0	1.0	1.0	1.0		1.0	0.069					0.99			0.806	1.0
	10.0			0.925	0.0	1.0		1.0	1.0	1.0	1.0		1.0	0.031					0.902			0.185	1.0
$s \leftarrow ps \leftarrow l$	0.01		1.0			1.0		1.0	1.0	1.0		1.0				1.0					1.0	1.0	
	0.1		1.0			1.0		1.0	1.0	1.0		1.0				1.0					1.0	1.0	
	1.0		1.0			1.0		1.0	1.0	1.0		1.0				1.0					1.0	1.0	
	10.0		1.0			1.0		1.0	1.0	1.0		1.0				1.0					1.0	0.999	
$p \leftarrow ps \leftarrow l$	0.01		1.0						1.0	0.948					1.0		1.0	0.98		1.0	1.0		
	0.1		1.0						1.0	0.551					1.0		1.0	0.98		1.0	0.999		
	1.0		1.0						1.0	0.003					1.0		1.0	0.74		1.0	0.995		
	10.0		1.0						1.0	0.0					1.0		1.0	0.22		1.0	0.961		

presents challenges in identifying a ground truth, which necessitates detecting and eliminating all entity integrity violations beforehand. Another promising research direction could involve examining datasets based on different data models [3, 15, 16, 25, 29]. Extending our approach to incorporate additional data quality [40] and big data dimensions [48] would further enrich our framework. These extensions could address both extrinsic and generic scenarios, as well as intrinsic and tailored ones [37]. Lastly, investigating other types of data integrity violations and applying our framework to these cases represents an interesting avenue for future research.

REFERENCES

[1] Ziawasch Abedjan. 2022. Enabling data-centric AI through data quality management and data literacy. *it Inf. Technol.* 64, 1-2 (2022), 67–70.

[2] Ziawasch Abedjan, Lukasz Golab, Felix Naumann, and Thorsten Papenbrock. 2018. *Data Profiling.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00878ED1V01Y201810DTM052

[3] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Keith W. Hare, Jan Hidders, Victor E. Lee, Bei Li, Leonid Libkin, Wim Martens, Filip Murlak, Josh Perryman, Ognjen Savkovic, Michael Schmidt, Juan F. Sequeda, Slawek Staworko, and Dominik Tomaszuk. 2021. PG-Keys: Keys for Property Graphs. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021.* 2423–2436.

[4] Marcelo Arenas, Leopoldo Bertossi, and Jan Chomicki. 1999. Consistent query answers in inconsistent databases. In *Proceedings of the Eighteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems* (Philadelphia, Pennsylvania, USA) *(PODS '99).* Association for Computing Machinery, New York, NY, USA, 68–79. https://doi.org/10.1145/303976.303983

[5] Donald P. Ballou and Giri Kumar Tayi. 1999. Enhancing Data Quality in Data Warehouse Environments. *Commun. ACM* 42, 1 (1999), 73–78.

[6] Carlo Batini. 2018. Data Quality Assessment. In *Encyclopedia of Database Systems, Second Edition.*

[7] Carlo Batini and Monica Scannapieco. 2016. *Data and Information Quality - Dimensions, Principles and Techniques.* Springer.

[8] Carlo Batini and Monica Scannapieco. 2016. *Data and Information Quality: Dimensions, Principles and Techniques.* Springer.

[9] Leopoldo E. Bertossi. 2011. *Database Repairing and Consistent Query Answering.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00379ED1V01Y201108DTM020

[10] Leopoldo E. Bertossi. 2018. Consistent Query Answering. In *Encyclopedia of Database Systems, Second Edition.*

[11] Leopoldo E. Bertossi. 2018. Database Repair. In *Encyclopedia of Database Systems, Second Edition.*

[12] Leopoldo E. Bertossi. 2019. Database Repairs and Consistent Query Answering: Origins and Further Developments. In *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS.* 48–58.

[13] Leopoldo E. Bertossi and Floris Geerts. 2020. Data Quality and Explainable AI. *ACM J. Data Inf. Qual.* 12, 2 (2020), 11:1–11:9.

[14] Philip Bohannon, Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2007. Conditional Functional Dependencies for Data Cleaning. In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007,* Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 746–755. https://doi.org/10.1109/ICDE.2007.367920

[15] Pieta Brown and Sebastian Link. 2017. Probabilistic Keys. *IEEE Trans. Knowl. Data Eng.* 29, 3 (2017), 670–682. https://doi.org/10.1109/TKDE.2016.2633342

[16] Peter Buneman, Susan B. Davidson, Wenfei Fan, Carmem S. Hara, and Wang Chiew Tan. 2001. Keys for XML. In *Proceedings of the Tenth International World Wide Web Conference, WWW 10, Hong Kong, China, May 1-5, 2001.* 201–210.

[17] Andrea Calì, Diego Calvanese, and Maurizio Lenzerini. 2013. Data Integration under Integrity Constraints. In *Seminal Contributions to Information Systems Engineering, 25 Years of CAiSE.* 335–352.

[18] Vassilis Christophides, Vasilis Efthymiou, and Kostas Stefanidis. 2015. *Entity Resolution in the Web of Data.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00655ED1V01Y201507WBE013

[19] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387.

[20] E. F. Codd. 1979. Extending the Database Relational Model to Capture More Meaning. *ACM Trans. Database Syst.* 4, 4 (1979), 397–434. https://doi.org/10.1145/320107.320109

[21] David W. Embley. 2018. Key. In *Encyclopedia of Database Systems, Second Edition.*

[22] Venkatesh Ganti and Anish Das Sarma. 2013. *Data Cleaning: A Practical Perspective.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00523ED1V01Y201307DTM036

[23] Ralf Hartmut Güting. 1994. An Introduction to Spatial Database Systems. *VLDB J.* 3, 4 (1994), 357–399.

[24] Miika Hannula and Jef Wijsen. 2022. A Dichotomy in Consistent Query Answering for Primary Keys and Unary Foreign Keys. In *PODS '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022.* 437–449.

[25] Vitaliy L. Khizder and Grant E. Weddell. 2003. Reasoning about Uniqueness Constraints in Object Relational Databases. *IEEE Trans. Knowl. Data Eng.* 15, 5 (2003), 1295–1306. https://doi.org/10.1109/TKDE.2003.1232279

[26] Monique F Kilkenny and Kerin M Robinson. 2018. Data quality: "Garbage in – garbage out". *Health Information Management Journal* 47, 3 (2018), 103–105. https://doi.org/10.1177/1833358318774357

[27] Paraschos Koutris, Xiating Ouyang, and Jef Wijsen. 2024. Consistent Query Answering for Primary Keys on Rooted Tree Queries. *Proc. ACM Manag. Data* 2,

2 (2024), 76.

[28] Paraschos Koutris and Jef Wijsen. 2016. Consistent Query Answering for Primary Keys. *SIGMOD Rec.* 45, 1 (2016), 15–22.

[29] Georg Lausen. 2007. Relational Databases in RDF: Keys and Foreign Keys. In *Semantic Web, Ontologies and Databases, VLDB Workshop, SWDB-ODBIS 2007, Vienna, Austria, September 24, 2007, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 5005)*, Vassilis Christophides, Martine Collard, and Claudio Gutierrez (Eds.). Springer, 43–56.

[30] Alexander W. Lee, Justin Chan, Michael Fu, Nicolas Kim, Akshay Mehta, Deepti Raghavan, and Ugur Çetintemel. 2025. Semantic Integrity Constraints: Declarative Guardrails for AI-Augmented Data Processing Systems. *Proc. VLDB Endow.* 18, 11 (2025), 4073–4080. https://doi.org/doi:10.14778/3749646.3749677

[31] Claudio L. Lucchesi and Sylvia L. Osborn. 1978. Candidate Keys for Relations. *J. Comput. Syst. Sci.* 17, 2 (1978), 270–279. https://doi.org/10.1016/0022-0000(78)90009-0

[32] Mathilde Marcy, Jean-Marc Petit, Marian Scuturici, Jocelyn Bonjour, Camille Fertel, and Gérald Cavalier. 2025. Can Surrogate Keys Negatively Impact Data Quality? *Proc. VLDB Endow.* 18, 12 (2025), 5279–5282.

[33] Sedir Mohammed, Lisa Ehrlinger, Hazar Harmouch, Felix Naumann, and Divesh Srivastava. 2025. The Five Facets of Data Quality Assessment. *SIGMOD Rec.* 54, 2 (2025), 18–27.

[34] Felix Naumann and Melanie Herschel. 2010. *An Introduction to Duplicate Detection.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00262ED1V01Y201003DTM003

[35] Thomas C. Redman. 1998. The impact of poor data quality on the typical enterprise. *Commun. ACM* 41, 2 (Feb. 1998), 79–82. https://doi.org/10.1145/269012.269025

[36] Shazia Sadiq (Ed.). 2013. *Handbook of Data Quality, Research and Practice.* Springer. https://doi.org/10.1007/978-3-642-36257-6

[37] Shazia Sadiq, Tamraparni Dasu, Xin Luna Dong, Juliana Freire, Ihab F. Ilyas, Sebastian Link, Renée J. Miller, Felix Naumann, Xiaofang Zhou, and Divesh Srivastava. 2017. Data Quality: The Role of Empiricism. *SIGMOD Rec.* 46, 4 (2017), 35–43. https://doi.org/10.1145/3186549.3186559

[38] Shazia Wasim Sadiq, Tamraparni Dasu, Xin Luna Dong, Juliana Freire, Ihab F. Ilyas, Sebastian Link, Renée J. Miller, Felix Naumann, Xiaofang Zhou, and Divesh Srivastava. 2017. Data Quality: The Role of Empiricism. *SIGMOD Rec.* 46, 4 (2017), 35–43. https://doi.org/10.1145/3186549.3186559

[39] Barna Saha and Divesh Srivastava. 2014. Data quality: The other face of Big Data. In *IEEE 30th International Conference on Data Engineering, Chicago, ICDE.* 1294–1297.

[40] Laura Sebastian-Coleman. 2013. *Measuring Data Quality for Ongoing Improvement: A Data Quality Assessment Framework.* MK Series on Business Intelligence.

[41] Rohit Singh, Venkata Vamsikrishna Meduri, Ahmed K. Elmagarmid, Samuel Madden, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Armando Solar-Lezama, and Nan Tang. 2017. Synthesizing Entity Matching Rules by Examples. *Proc. VLDB Endow.* 11, 2 (2017), 189–202. https://doi.org/10.14778/3149193.3149199

[42] Bernhard Thalheim. 1991. *Dependencies in relational databases.* Vieweg + Teubner Verlag.

[43] Bernhard Thalheim. 2000. *Entity-relationship modeling - foundations of database technology.* Springer.

[44] Richard Y. Wang and Diane M. Strong. 1996. Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems* 12, 4 (1996), 5–33.

[45] Jef Wijsen. 1993. A Theory of Keys for Temporal Databases. In *Neuvièmes Journées Bases de Données Avancées, 27-30 Septembre 1993, Toulouse, France (Informal Proceedings).* 35–54.

[46] Jef Wijsen. 2014. A Survey of the Data Complexity of Consistent Query Answering under Key Constraints. In *Foundations of Information and Knowledge Systems - 8th International Symposium, FoIKS.* 62–78.

[47] Wang Ngai Yeung, Riccardo Di Clemente, and Renaud Lambiotte. 2025. Garbage in garbage out? Impacts of data quality on criminal network intervention. *EPJ Data Sci.* 14, 1 (2025), 37.

[48] Shen Yin and Okyay Kaynak. 2015. Big Data for Modern Industry: Challenges and Trends [Point of View]. *Proc. IEEE* 103, 2 (2015), 143–146. https://doi.org/10.1109/JPROC.2015.2388958

[49] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Ben Hu. 2025. Data-centric Artificial Intelligence: A Survey. *ACM Comput. Surv.* 57, 5 (2025), 129:1–129:42.